INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	CONSTRUCCION DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	*Arquitectura como modelo: el modelo C4 (contexto, contenedores, componentes, dinámico y despliegue) con Structurizr DSL, C4-PlantUML, Mermaid e IA*				
NÚMERO DE PRÁCTICA:	*06*	**AÑO LECTIVO:**	*2026B*	**NRO. SEMESTRE:**	*VI*
FECHA DE PRESENTACIÓN	*17/10/26*	**HORA DE PRESENTACIÓN**			
INTEGRANTE (s): Oliva Valdivia David Alexander				**NOTA:**	
DOCENTE(s): *Mg. Antonio Arroyo Paz*					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS RESUELTOS

Repositorio: `github.com/DavidOliva06/cs-2026b-lab04-grupo06`, carpeta `docs/architecture/c4/`. La entrega del 17/10/26 terminó en el commit `d8c2792`, sin etiqueta y sin la sección del README. Esta versión del informe corresponde a la etiqueta `v0.6-c4` (commit `2a145ac`, rama `lab06-correcciones`), que agrega lo que faltaba y las correcciones de la revisión (sección II). Todas las rutas son relativas a la raíz del repositorio.

Tabla 1. Commits que respaldan cada ejercicio.

Ejercicio	Commit	Mensaje	Fecha
E1	`59ef46b`	E1: contexto C4.	2026-10-17
E2	`c31a279`	E2: contenedores C4.	2026-10-17
E3	`4baf2fd`	E3: componentes C4.	2026-10-17
E4	`c327c2b`	E4:Vista dinámica del flujo crítico	2026-10-17
E5	`8ef5125`	E5:Vista de despliegue	2026-10-17
E6	`d75c8da`	E6:Exportación y comparación de herramientas	2026-10-17
E7 y E8	`d8c2792`	README, revisión cruzada y entrega (no modificó el README)	2026-10-17
Revisión	`2a145ac`	Lab 06: correcciones tras la revisión del informe (etiqueta `v0.6-c4`)	2026-11-13

E1: Inventario y diagrama de contexto

1. **Cree `docs/architecture/c4/inventario.md` con la tabla del Paso 1: cada persona, sistema, contenedor y componente, con su tipo C4 y el archivo de los Laboratorios 04 o 05 de donde proviene.**

`docs/architecture/c4/inventario.md` tiene 17 filas. Las de los niveles 1 a 3 son:

Tabla 2. Inventario de elementos C4 (niveles 1 a 3).

Elemento	Tipo C4	Origen
Ciudadano, brigadista, coordinador	Persona	drivers.md (Lab 04)
SismoReporta AQP	Sistema de software	Caso (README.md)
Servicio de teselas de mapa (OpenStreetMap o proveedor)	Sistema externo	Diagrama de arquitectura (Lab 04)
Aplicación web progresiva (PWA), con la cola local en IndexedDB	Contenedor	ADR-003 (Lab 04)
API (monolito modular en Python)	Contenedor	ADR-001 (Lab 04)
Worker en segundo plano	Contenedor	ADR-001 (Lab 04)
Base de datos PostgreSQL, con la cola de trabajos	Contenedor	ADR-001 y ADR-002 (Lab 04)
Almacenamiento de fotos en disco	Contenedor	Diagrama de arquitectura (Lab 04)
Proxy inverso	Nodo de infraestructura (despliegue)	Diagrama de arquitectura (Lab 04)
Módulos Reportes, Mapa, Priorización, Brigadas y Compartido	Componente (de la API)	ADR-001 y paquetes.puml (Lab 05)

Las otras filas registran los componentes de la PWA, los puertos, las clases (nivel 4), los dos flujos candidatos a vista dinámica y los nodos de despliegue. Criterios de clasificación del archivo: IndexedDB es parte de la PWA porque vive en el navegador; la cola de trabajos es una tabla de PostgreSQL (ADR-002); API y worker son contenedores distintos porque se ejecutan como procesos separados (ADR-001).

2. Adapte el Prompt IA 1 con su inventario y su ADR-001 para obtener un borrador de workspace.dsl. Registre el prompt en la bitácora.

El prompt se adaptó con `inventario.md` y el ADR-001, y pidió tres cosas: identificadores jerárquicos (`!identifiers hierarchical`), solo las tecnologías del ADR-001 y no confundir un contenedor C4 con uno de Docker. Está registrado como interacción 1 de `docs/architecture/c4/bitacora-ia.md`, junto con la tabla de revisión del borrador. El texto literal del prompt no se guardó; la bitácora lo indica.

3. Revise el borrador con el Paso 2 como guía. Corrija en especial: contenedores que en realidad son infraestructura, sistemas externos modelados como internos y tecnologías que no están en su ADR.

Tabla 3. Revisión del borrador de la IA (bitácora, interacción 1): 4 corregidas, 1 rechazada y 7 aceptadas.

Propuesta de la IA	Decisión	Justificación
"Proxy inverso" como contenedor del sistema	Corregida	Es infraestructura que solo se configura: pasa a nodo de infraestructura en la vista de despliegue.
Tecnología "Por definir (fuera del ADR-001)" para el proxy	Corregida	No figura en el ADR-001; elegir el producto requiere su propio ADR.
Relaciones PWA → proxy → API en la vista de contenedores	Corregida	En contenedores, la PWA habla con la API "a través del proxy inverso"; el proxy solo aparece en despliegue.
"1 vCPU y 2 GB de RAM" en el servidor	Corregida	El dato venía de despliegue.py, no del ADR-001; el tamaño se fijó después en E5 con su costo.
Nginx, Gunicorn, Prometheus y Grafana (de despliegue.py)	Rechazada	No figuran en el ADR-001 ni en R-02.
"Servicio de teselas de mapa" con la etiqueta "Externo"	Aceptada	No lo construimos ni lo desplegamos: está fuera del límite del sistema.
Almacenamiento de fotos como contenedor; worker separado de la API	Aceptadas	Un sistema de archivos que guarda datos es un contenedor; el worker es otro proceso (ADR-001).

4. Ejecute structurizr.sh validate hasta que no haya errores. Haga commit con el mensaje E1: contexto C4.

Contexto del modelo (`docs/architecture/c4/workspace.dsl`, líneas 19-23, 43 y 46-52, versión `v0.6-c4`):

```
ciudadano = person "Ciudadano" "Reporta daños con foto y ubicación tras un sismo."
brigadista = person "Brigadista" "Atiende las zonas asignadas y actualiza su estado."
coordinador = person "Coordinador" "Revisa el mapa de daños, prioriza zonas y asigna brigadas."
sismo = softwareSystem "SismoReporta AQP" "Recibe reportes de daños, prioriza zonas y asigna brigadas." { ... }
teselas = softwareSystem "Servicio de teselas de mapa" "Mapa base: ..." "Externo"

ciudadano -> sismo.pwa "Envía reportes de daños con foto y ubicación" "HTTPS"
sismo.pwa -> teselas "Obtiene el mapa base" "HTTPS/PNG"

$ structurizr.sh validate -w docs/architecture/c4/workspace.dsl
$ echo $?
0
```

`validate` termina sin mensajes y con código 0 (Structurizr CLI 2025.11.09). Las relaciones de las personas apuntan a la PWA; la vista de contexto las sube al sistema. Commit `59ef46b` "E1: contexto C4.".

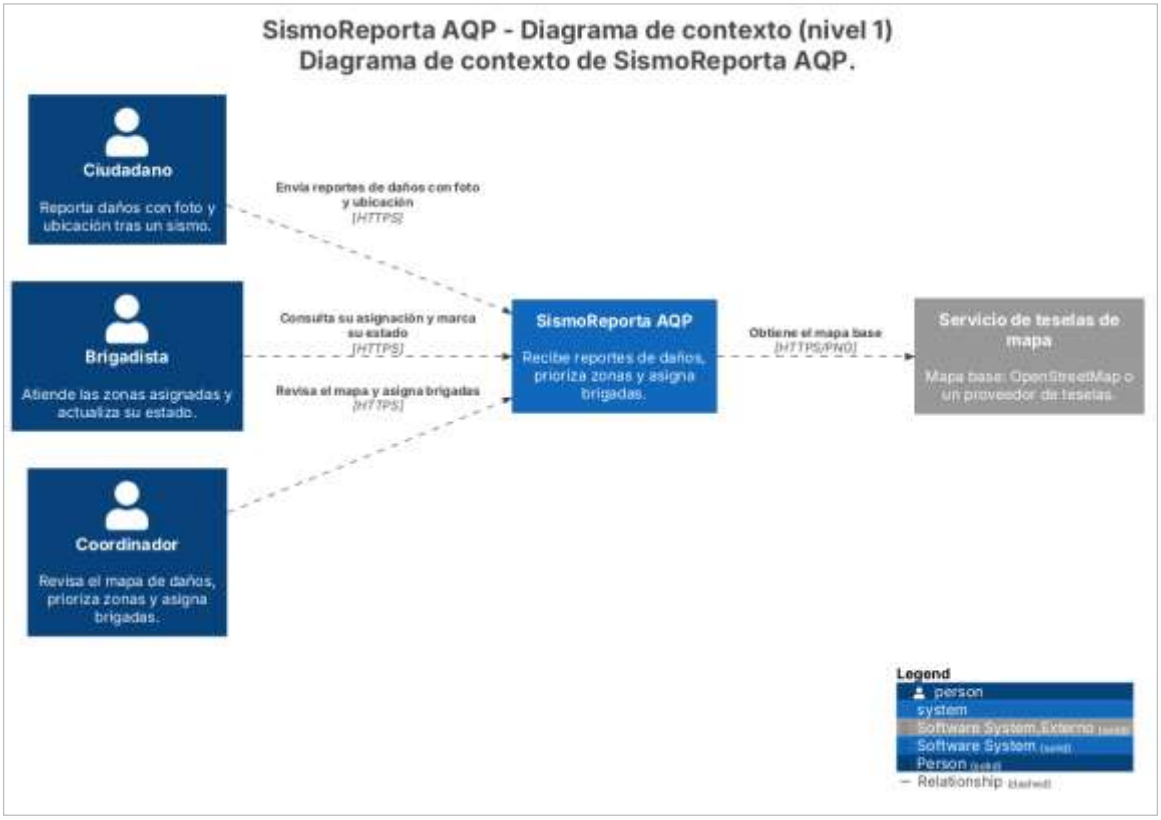


Figura 1. Diagrama de contexto exportado a C4-PlantUML (img/structurizr-Contexto.png).

E2: Diagrama de contenedores

1. Agregue dentro del sistema 3 o más contenedores (aplicaciones y almacenes de datos), cada uno con descripción y tecnología concreta.

Tabla 4. Contenedores de SismoReporta AQP (workspace.dsl, líneas 25-40).

Contenedor	Tecnología	Forma	Descripción (resumen)
Aplicación web progresiva	JavaScript, CSS, Service Worker e IndexedDB	Web Browser	Interfaz de los tres roles; cola local sin señal (ADR-003).

API	Python	—	Monolito modular: ingesta idempotente, mapa, priorización y brigadas (ADR-001).
Worker	Python	—	Procesa fotos y recalcula la priorización de forma periódica.
Base de datos	PostgreSQL	Database	Reportes, usuarios, consentimientos, zonas, asignaciones y la tabla de trabajos (ADR-002).
Almacenamiento de fotos	Sistema de archivos	Database	Fotos comprimidas y sin metadatos EXIF, en un volumen separado.

2. Agregue las relaciones entre contenedores y con los sistemas externos, indicando el protocolo (HTTPS/JSON, SQL/TCP, MQTT, etc.).

Tabla 5. Relaciones de la vista de contenedores (9; ninguna sin protocolo).

Origen → destino	Intención	Protocolo
Ciudadano, Brigadista, Coordinador → PWA	Enviar reportes / consultar la asignación / revisar el mapa	HTTPS
PWA → API	Envía reportes y consultas, a través del proxy inverso	HTTPS/JSON
PWA → Servicio de teselas	Obtiene el mapa base	HTTPS/PNG
API → Base de datos	Lee y escribe datos	SQL/TCP
API → Almacenamiento de fotos	Guarda la foto original	E/S de archivos
Worker → Base de datos	Toma trabajos con SKIP LOCKED y guarda las zonas priorizadas	SQL/TCP
Worker → Almacenamiento de fotos	Redimensiona las fotos y elimina los metadatos EXIF	E/S de archivos

3. Use etiquetas y estilos (Database, Queue, Web Browser) para que la forma de cada elemento comunique su tipo.

```
element "Database"     { shape Cylinder }
element "Web Browser"  { shape WebBrowser }
element "Externo"      { background #999999  color #ffffff }
element "Nodo externo" { background #eeeeee  color #444444  stroke #6b6b6b  border dashed }
```

No se usa Queue: la cola local vive en IndexedDB dentro de la PWA (ADR-003) y la cola de trabajos es una tabla de PostgreSQL (ADR-002), así que ninguna es un contenedor propio (comentario de las líneas 15-16 del workspace). El estilo "Nodo externo" se agregó en la revisión (sección II).

4. Defina la vista container, valide y exporte.

5. Haga commit con el mensaje E2: contenedores C4.

Vista `container sismo "Contenedores"` (línea 147), validada y exportada a `export/` (E6). Commit `c31a279` "E2: contenedores C4.".

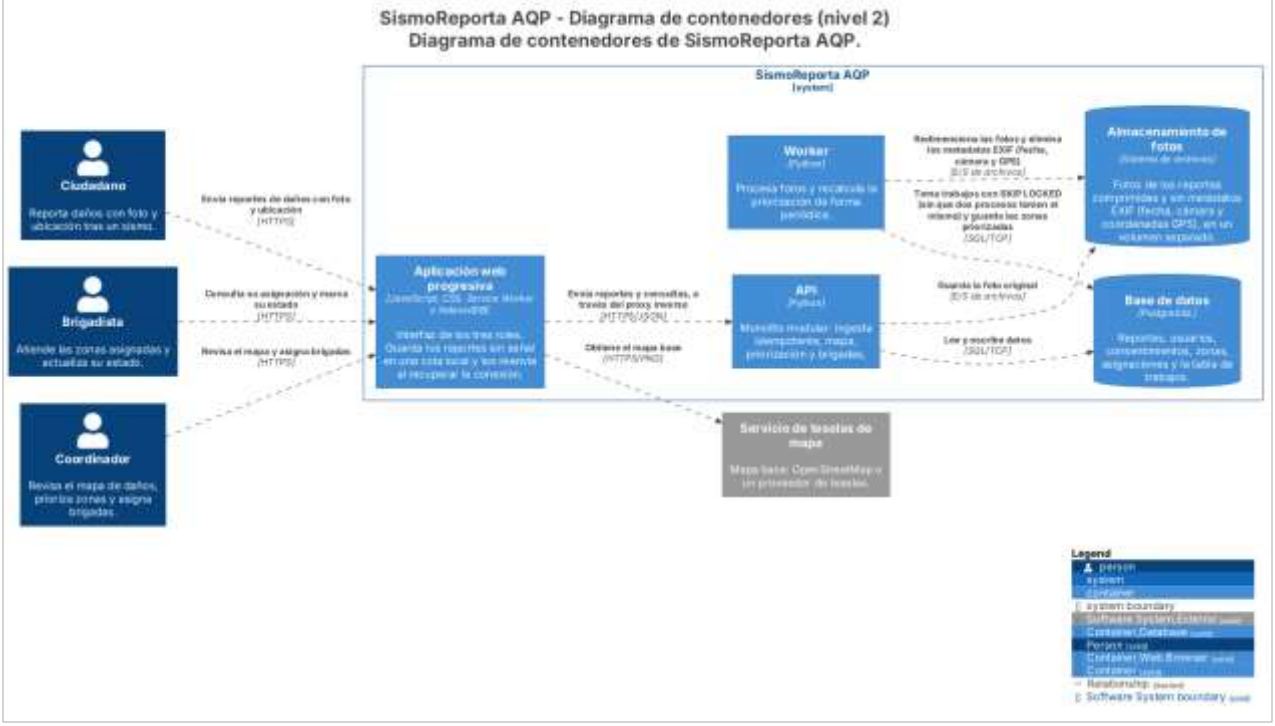


Figura 2. Diagrama de contenedores (img/structurizr-Contenedores.png).

E3: Diagrama de componentes

1. Elija el contenedor principal (normalmente la API) y agregue 4 o más componentes, que deben coincidir con los paquetes de docs/design/paquetes.puml del Laboratorio 05.

Tabla 6. Componentes de la API y paquete de origen.

Componente (workspace.dsl)	Paquete (paquetes.puml)	Contenido
Compartido (línea 29)	compartido	`Ubicacion`, `Zona` y el puerto `ColaTrabajos`.
Reportes (línea 30)	reportes	`ServicioIngesta` (ingesta idempotente por UUID), `AlmacenServidorReportes` y `ConsultaReportes`.
Priorización (línea 31)	priorización	`ZonaPriorizada` y el puerto `ConsultaZonasAsignadas` (ADR-004).
Brigadas (línea 32)	brigadas	`Brigada`, `Asignacion` y `ServicioAsignacion`, que implementa el puerto.
Mapa (línea 33)	mapa	`ServicioMapa`.

2. Modele las dependencias entre componentes y hacia contenedores o sistemas externos. Verifique que no haya ciclos, igual que en la regla C4 del Laboratorio 05.

Las 9 dependencias entre componentes copian las de `paquetes.puml` (líneas 40-54) y hay 6 más hacia la base de datos y el disco. La comprobación se automatizó en `docs/architecture/c4/verificar_c4.py`, que lee el modelo exportado a JSON:

```
$ structurizr.sh export -w docs/architecture/c4/workspace.dsl -f json -o /tmp/c4
$ python docs/architecture/c4/verificar_c4.py /tmp/c4/workspace.json docs/design/paquetes.puml
## 2. Componentes de la API frente a paquetes.puml
- Paquetes:    ['brigadas', 'compartido', 'mapa', 'priorizacion', 'reportes']
- Componentes: ['brigadas', 'compartido', 'mapa', 'priorizacion', 'reportes']
- ¿Mismos nombres? Sí
- Dependencias: 9 en paquetes.puml, 9 en el modelo; faltan 0, sobran 0
## 3. Ciclos entre componentes
```

```
- Sin ciclos
Comprobaciones fallidas: 0
```

Para comprobar que el script detecta un ciclo, se agregó en una copia la relación `compartido → mapa`: el script respondió "Ciclo: compartido -> mapa -> priorizacion -> reportes -> compartido" y terminó con código 1. El orden sin ciclos es el de la regla del Lab 05: mapa y brigadas → priorización → reportes → compartido.

3. Defina la vista component, valide y haga commit con el mensaje E3: componentes C4.

Vista `component sismo.api "Componentes"` (línea 153). Commit `4baf2fd` "E3: componentes C4.".

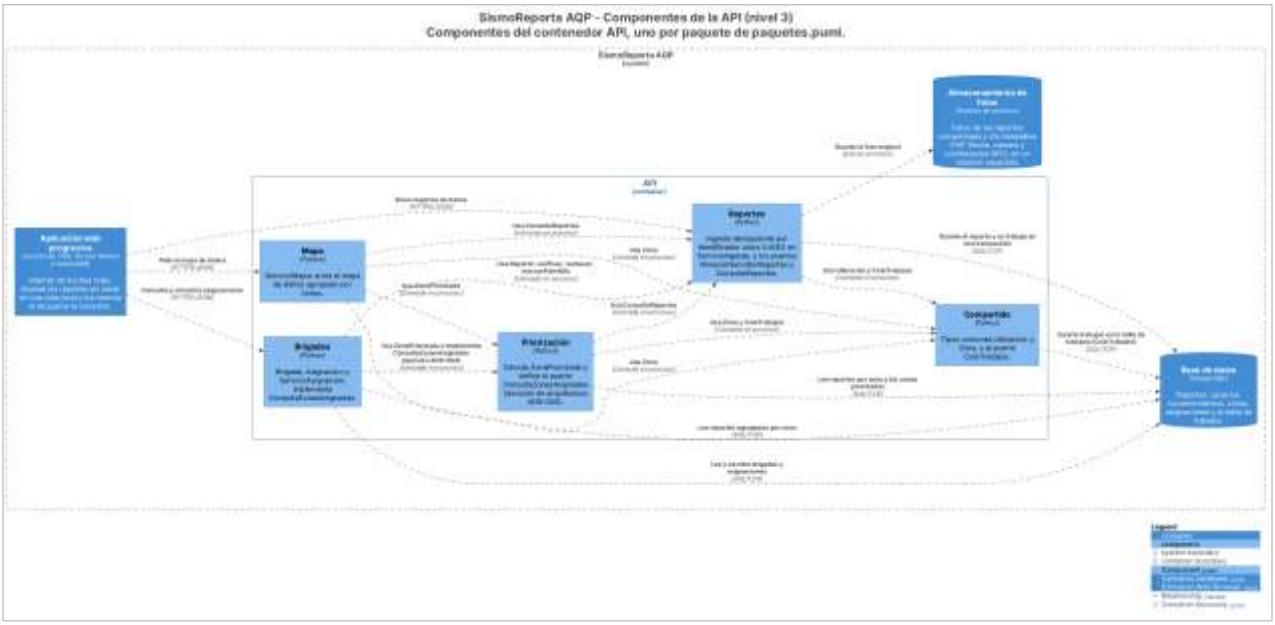


Figura 3. Componentes del contenedor API (img/structurizr-Componentes.png).

E4: Vista dinámica del flujo crítico

1. Tome el mismo flujo del diagrama de secuencia del Laboratorio 05.

Flujo: HU-01, **enviar un reporte sin señal y sincronizarlo**, de `docs/design/secuencia-reporte.puml`.

2. Escriba una vista dynamic con 4 o más pasos entre contenedores o componentes; cada paso debe corresponder a una relación ya existente en el modelo.

```
dynamic sismo.api "SincronizarReporte" "HU-01: enviar un reporte sin señal y sincronizarlo." {
    ciudadano -> sismo.pwa "Registra el reporte con foto y ubicación; sin señal queda en la cola local"
    sismo.pwa -> sismo.api.reportes "Al recuperar la señal, envía cada reporte pendiente con su ... (UUID)"
    sismo.api.reportes -> sismo.bd "Guarda el reporte solo si su UUID no existe (guardarSiNoExiste)"
    sismo.api.reportes -> sismo.fotos "Guarda la foto original"
    sismo.api.reportes -> sismo.api.compartido "Encola el procesamiento de la foto mediante ColaTrabajos"
    sismo.api.compartido -> sismo.bd "Inserta el trabajo en la misma transacción que el reporte"
}
```

Tabla 7. Cada paso, su relación en el modelo y el mensaje de la secuencia del Lab 05.

Paso	Relación del modelo (línea)	Mensaje de secuencia-reporte.puml
1	ciudadano → pwa (46)	Líneas 20-35: "Enviar", `crearReporte`, `encolar`, `guardar` en la cola local.
2	pwa → api.reportes (59)	Línea 53: `ENVIO -> ING: recibir(reporte)`.
3	api.reportes → bd (79)	Línea 54: `ING -> SERV: guardarSiNoExiste(reporte)`.
4	api.reportes → fotos (80)	No aparece: la secuencia no detalla dónde se guarda la foto.

5	api.reportes → api.compartido (76)	No aparece: la secuencia termina en `guardarSiNoExiste`.
6	api.compartido → bd (84)	No aparece.

Structurizr rechaza un paso que no tenga relación en el modelo, así que `validate` sin errores prueba que los 6 pasos la tienen. Los pasos 4 a 6 muestran almacenes y la transacción del servidor que la secuencia no dibuja.

3. **En comparacion.md, explique en 3 a 5 líneas qué detalle muestra la secuencia UML que la vista dinámica omite, y para qué público sirve cada una.**

Primera parte de `docs/architecture/c4/comparacion.md` (4 líneas):

- La secuencia muestra lo que la vista dinámica omite: la rama de error por consentimiento no aceptado (`alt`), el ciclo por cada reporte pendiente (`loop`), la condición de confirmación (`opt`), los mensajes de retorno y las clases internas de la PWA con sus operaciones y parámetros.
- La vista dinámica solo numera el camino feliz entre contenedores y componentes, pero deja ver a qué almacenes llega el reporte y que el trabajo se encola en la misma transacción.
- La secuencia sirve a los desarrolladores que implementan y prueban la HU-01.
- La vista dinámica sirve a arquitectos, operadores y nuevos integrantes que necesitan saber qué partes recorre el flujo sin entrar al código.

Commit `c327c2b` "E4:Vista dinámica del flujo crítico".

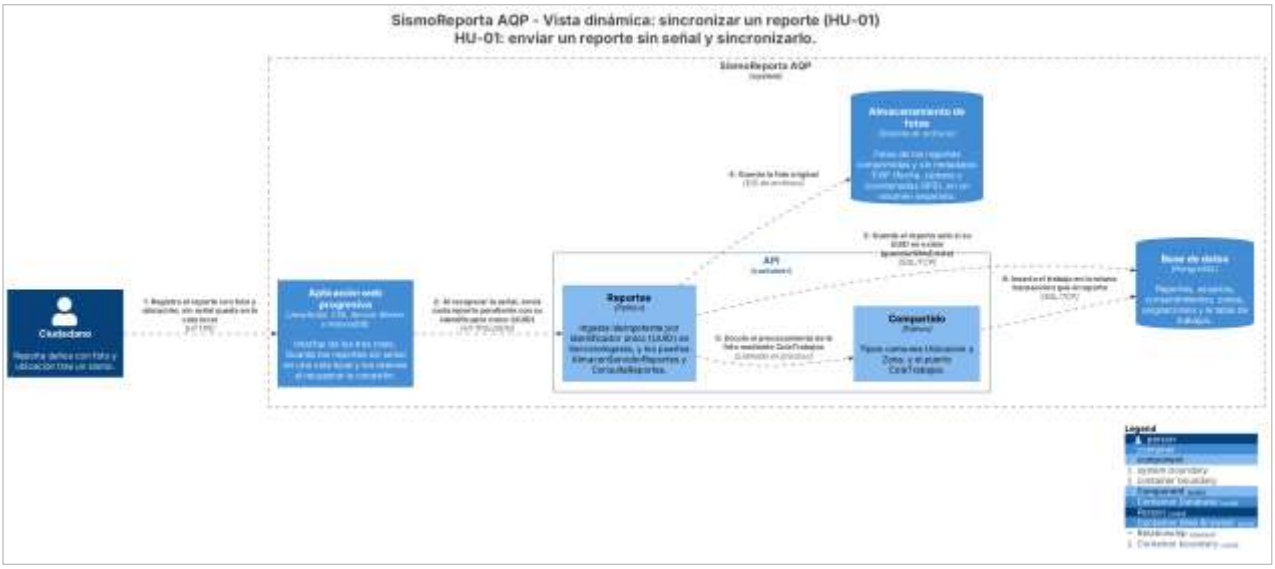


Figura 4. Vista dinámica SincronizarReporte (img/structurizr-SincronizarReporte.png).

E5: Vista de despliegue

1. **Defina deploymentEnvironment "Producción" con nodos anidados: dispositivo del usuario, servidor o nube, Docker u otro entorno de ejecución.**

2. **Coloque una instancia (containerInstance) de cada contenedor del nivel 2 en algún nodo.**

3. **Indique sistema operativo, tamaño o servicio de nube de cada nodo, y compruebe que sea coherente con la restricción de presupuesto de su drivers.md.**

Tabla 8. Nodos del entorno Producción (workspace.dsl, líneas 91-126) e instancias.

Nodo (anidamiento)	SO, tamaño o servicio	Instancias

Celular del ciudadano o del brigadista › Navegador web	Android › Chrome para Android	PWA
Computadora del coordinador › Navegador web	Windows o Linux › Chrome, Edge o Firefox	PWA
DigitalOcean › Droplet Basic	Región Nueva York 3 (NYC3) › Ubuntu 24.04 LTS; 1 vCPU, 2 GiB, 50 GiB SSD	Proxy inverso (nodo de infraestructura)
Droplet › Entorno de ejecución Python	Python 3.12 y systemd, sin Docker	API y Worker
Droplet › Servidor PostgreSQL	PostgreSQL 16 en el mismo droplet	Base de datos
Droplet › Volumen de bloques	DigitalOcean Volumes, ext4, 100 GiB	Almacenamiento de fotos
Proveedor de teselas (externo)	Internet	Servicio de teselas de mapa

Salida de `verificar_c4.py` (comprobación 4): los cinco contenedores tienen instancia; la PWA tiene dos, una en cada navegador. No hay Docker: el entorno de ejecución son dos servicios de systemd, coherente con la respuesta 2 del cuestionario.

Tabla 9. Costo mensual frente a R-03 ("un solo servidor... aprox. US$ 20-30/mes").

Concepto	**Base del cálculo**	**US$/mes**
Droplet Basic 1 vCPU / 2 GiB	Precio de lista de DigitalOcean	12,00
Respaldos diarios	30 % del droplet	3,60
Volumen de bloques de 100 GiB	US$ 0,10 por GiB	10,00
Total	Un solo servidor, sin servicios administrados (R-03)	**25,60**

US$ 25,60 está dentro del rango de R-03 y deja US$ 4,40 de margen. Commit `8ef5125` "E5:Vista de despliegue".

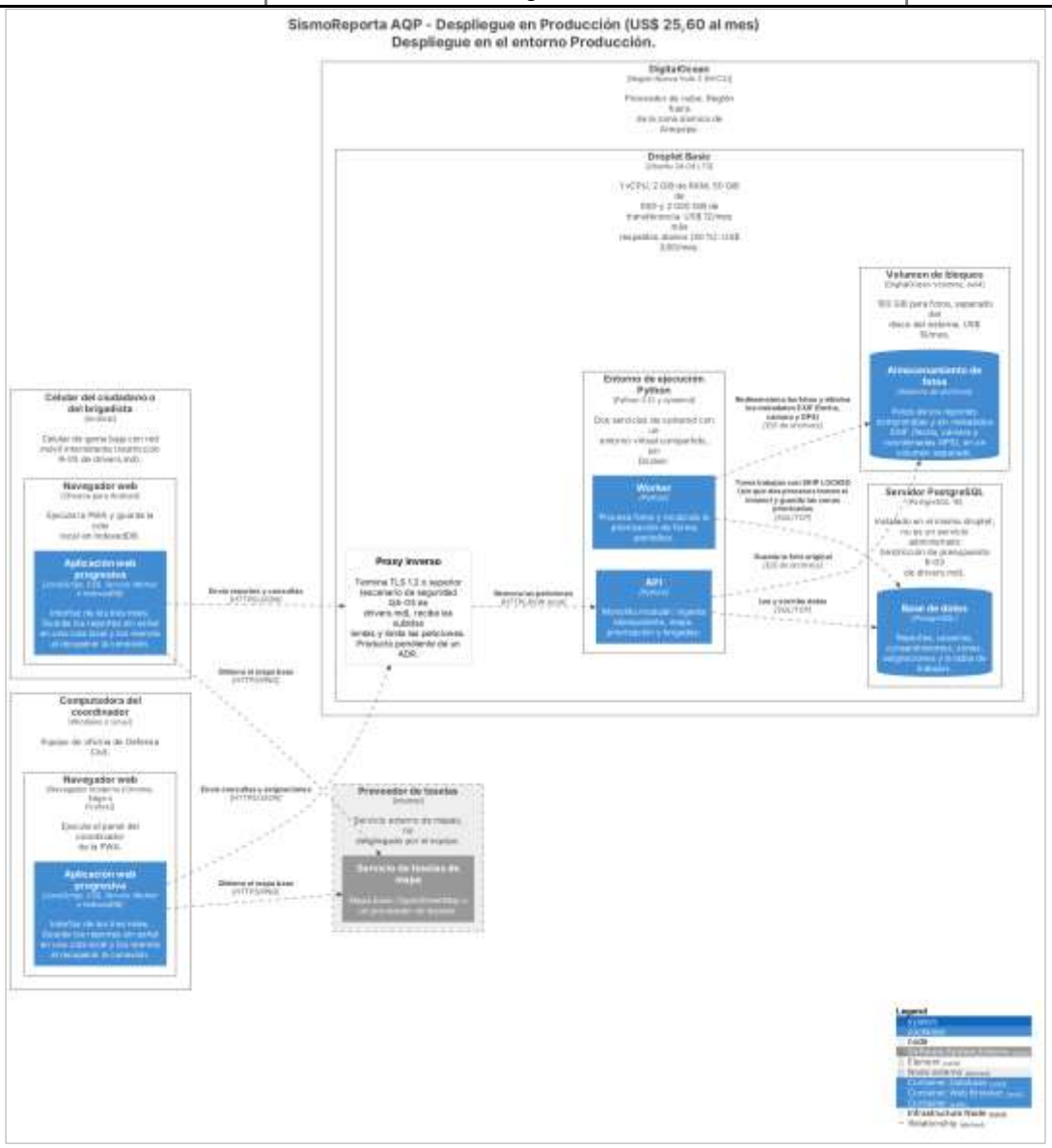


Figura 5. Despliegue en Producción (img/structurizr-Despliegue.png).

E6: Exportación y comparación de herramientas

1. **Exporte todas las vistas a C4-PlantUML y a Mermaid en la carpeta export/, y genere los PNG en img/.**

```
$ cd docs/architecture/c4
$ structurizr.sh export -w workspace.dsl -f plantuml/c4plantuml -o export
$ structurizr.sh export -w workspace.dsl -f mermaid -o export
$ cd export && java -jar plantuml.jar -tpng -charset UTF-8 -o ../img *.puml
```

Resultado: 10 archivos en `export/` (`structurizr-<Vista>.puml` y `.mmd` para Contexto, Contenedores, Componentes, SincronizarReporte y Despliegue) y 5 PNG en `img/`, que son las Figuras 1 a 5. En la revisión se volvieron a exportar con la propiedad `"c4plantuml.tags" "true"` (línea 137), sin editar a mano ningún archivo exportado.

2. **Escriba a mano contexto-manual.puml con las macros de C4-PlantUML (como en la sección 1.5) para el mismo contexto.**

```
@startuml contexto-manual
```

```
!include <C4/C4_Context>
LAYOUT_LEFT_RIGHT()
title SismoReporta AQP - Diagrama de contexto
Person(ciudadano, "Ciudadano", "Reporta daños con foto y ubicación tras un sismo.")
Person(brigadista, "Brigadista", "Atiende las zonas asignadas y actualiza su estado.")
Person(coordinador, "Coordinador", "Revisa el mapa de daños, prioriza zonas y asigna brigadas.")
System(sismo, "SismoReporta AQP", "Recibe reportes de daños, prioriza zonas y asigna brigadas.")
System_Ext(teselas, "Servicio de teselas de mapa", "Mapa base: OpenStreetMap o un proveedor de teselas.")
Rel(ciudadano, sismo, "Envía reportes de daños con foto y ubicación", "HTTPS")
Rel(brigadista, sismo, "Consulta su asignación y marca su estado", "HTTPS")
Rel(coordinador, sismo, "Revisa el mapa y asigna brigadas", "HTTPS")
Rel(sismo, teselas, "Obtiene el mapa base", "HTTPS/PNG")
SHOW_LEGEND()
@enduml
```

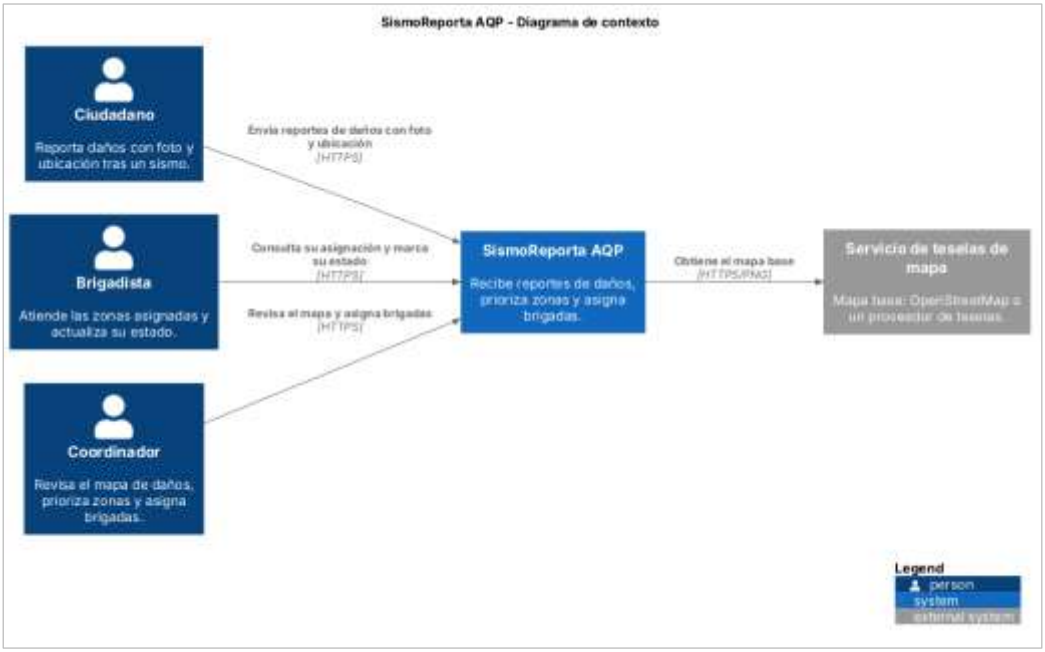


Figura 6. Contexto escrito a mano con macros de C4-PlantUML (contexto-manual.puml).

3. En comparacion.md, compare ambos enfoques (escritura a mano frente a modelo exportado) en: tiempo invertido, errores encontrados y qué pasaría si mañana cambia el nombre de un contenedor.

Tabla 10. Resumen de la segunda parte de comparacion.md.

Criterio	A mano (contexto-manual.puml)	Modelo exportado (workspace.dsl)
Tiempo y esfuerzo	24 líneas para una sola vista; cada vista adicional se escribe aparte.	201 líneas en la entrega para las cinco vistas; los 10 archivos de `export/` se generan con dos comandos. El tiempo en minutos no se registró.
Errores encontrados	Con `Rel(sismo, tesela, ...)` PlantUML no da error y dibuja un elemento suelto "tesela".	El mismo error se detiene: `The destination element "tesela" does not exist at line 52`. La vista dinámica solo acepta pasos sobre relaciones existentes.
Renombrar un contenedor ("API" → "Servidor de aplicaciones")	El contexto no cambia; en las demás vistas habría que buscar y cambiar cada `Container(...)` sin aviso si queda uno.	Cambia 1 línea; `validate` pasa y se vuelve a exportar. Los alias generados cambian en 0, 4, 25, 9 y 4 líneas de los `.puml` (Contexto, Contenedores, Componentes, Dinámica y Despliegue).
Defectos de presentación	`System_Ext` sale gris y el título está en español.	En la entrega: "Externo" se perdía y los títulos salían en inglés. Corregido con `c4plantuml.tags` y `title`.

Commit `d75c8da` "E6:Exportación y comparación de herramientas".

E7: Revisión, comunicación y bitácora

1. Aplique la lista de revisión oficial (Tabla 3) a sus cinco vistas en revision-c4.md (sí/no y evidencia).

`docs/architecture/c4/revision-c4.md` responde las 12 preguntas de la lista para cada vista, con evidencia (línea del DSL o salida de `verificar_c4.py`). Los conteos del script: 0 elementos sin descripción (5, 9, 8, 6 y 18 elementos por vista) y 0 relaciones sin descripción ni tecnología (4, 9, 18, 6 y 9).

Tabla 11. Resultado de la lista de revisión por vista.

Vista	Entrega (17/10): Sí / No	v0.6-c4: Sí / No	Qué cambió
Contexto	11 / 1	12 / 0	Título.
Contenedores	10 / 2	12 / 0	Título; EXIF y SKIP LOCKED explicados.
Componentes	10 / 2	12 / 0	Título; UUID y ADR-004 explicados.
Dinámica	10 / 2	12 / 0	Título; UUID explicado.
Despliegue	9 / 3	11 / 1	Título; NYC3, QA-03, R-03 y R-05 explicados. Sigue en "No": el proxy inverso no tiene tecnología porque ningún ADR elige el producto.

2. Use el Prompt IA 2 como segundo revisor e identifique al menos un falso positivo.

```
Actúa como revisor de diagramas de arquitectura. Evalúa estas vistas exportadas de un
modelo C4 con la lista de revisión de c4model.com: generales, elementos y relaciones.
Responde en una tabla (vista, pregunta de la lista, cumple sí/no, evidencia, corrección
sugerida). No inventes problemas: si algo cumple, indícalo.
```

La IA devolvió 22 filas (bitácora, interacción 6). Al verificarlas contra los archivos y los PNG se descartaron cinco:

Tabla 12. Hallazgos de la IA descartados.

Hallazgo de la IA	Tipo	Por qué no se sostiene
Dinámica en Mermaid: "No" en tipo y alcance	Falso positivo	El título dice "Dynamic View: ... - API": tipo y alcance se entienden; nombrar la HU-01 es una mejora, no lo que pregunta la lista.
Contenedores: "se pierde la forma de navegador de la PWA"	Falso positivo	La lista pregunta si las formas se entienden, no si se conserva una forma concreta; la leyenda explica el contenedor.
Despliegue en Mermaid: nombres de nodos cortados	Falso positivo	En el archivo los nombres están completos; el corte es de las fuentes del renderizador sin interfaz.
Acrónimos sin explicar en Contenedores y Dinámica	Exagerado	SKIP LOCKED no es un acrónimo, y UUID es estándar para un público técnico. Aun así se explicaron en la revisión.
Contexto en Mermaid: "No" en colores por falta de leyenda	Exagerado	Es el mismo problema de la fila "¿Tiene leyenda?", contado dos veces.

Dos hallazgos de la IA sí eran reales y se aplicaron en `v0.6-c4`: el servicio de teselas salía azul en C4-PlantUML (falta de `c4plantuml.tags`) y el nodo "Proveedor de teselas" salía blanco sobre blanco en Mermaid (estilo "Nodo externo").

3. Use el Prompt IA 3 para redactar explicacion.md (120 palabras o menos) y corríjala: verifique que todo lo que dice sea cierto según su modelo.

Versión final de `docs/architecture/c4/explicacion.md` (115 palabras):

> *"SismoReporta AQP es una aplicación que ayuda a Defensa Civil a organizar la respuesta después de un sismo en Arequipa. La usan tres tipos de personas. Los ciudadanos envían desde su celular una foto y la ubicación de los daños que ven; si no hay señal, el reporte se guarda y se envía solo cuando vuelve la*

conexión. Los coordinadores ven los reportes en un mapa, saben qué zonas necesitan ayuda primero y envían brigadas. Los brigadistas consultan la zona que deben atender y marcan en la aplicación cómo va su atención. El sistema depende de un único servicio externo: un proveedor de mapas que dibuja las calles sobre las que se muestran los daños."

Cada afirmación se contrastó con el modelo (tabla en la bitácora, interacción 7). Cinco se mantuvieron y dos se corrigieron:

Tabla 13. Frases del borrador de la IA que el modelo no respalda.

Borrador de la IA	Qué dice el modelo	Versión corregida
"Los coordinadores ven **todos** los reportes en un mapa"	"Revisa el mapa de daños": no dice que muestre todos.	"ven los reportes en un mapa"
"Los brigadistas... avisan cuando **van en camino y cuando terminan**"	"Consulta su asignación y marca su estado". Los estados "en camino" y "atendido" son de RF-05 (`drivers.md`), no del modelo C4.	"consultan la zona que deben atender y marcan en la aplicación cómo va su atención"

4. Complete bitacora-ia.md con 5 o más interacciones (fecha, herramienta, prompt, propuesta, verificación, decisión).

`docs/architecture/c4/bitacora-ia.md` abre con una tabla de 8 interacciones con esas seis columnas (herramienta: Claude, de Anthropic) y detalla cada una en su sección. En la entrega tenía cinco secciones sin fecha, herramienta ni verificación; se completaron en `v0.6-c4`.

Tabla 14. Interacciones de la bitácora.

#	Fecha	Ejercicio	Decisión
1	2026-10-17	E1: borrador de workspace.dsl	4 corregidas, 1 rechazada y 7 aceptadas (Tabla 3).
2	2026-10-17	E2: contenedores y formas	`Database` y `Web Browser`; ningún contenedor con Queue.
3	2026-10-17	E3: componentes	Se acepta: 5 componentes y 9 dependencias sin ciclos.
4	2026-10-17	E4: vista dinámica	Se acepta: 6 pasos sobre relaciones existentes.
5	2026-10-17	E5: despliegue	Se acepta: US$ 25,60/mes dentro de R-03.
6	2026-10-17	E7: Prompt IA 2	5 hallazgos descartados; 2 aplicados en la revisión.
7	2026-10-17	E7: Prompt IA 3	2 frases corregidas (Tabla 13).
8	2026-11-13	Revisión del informe	Rama `lab06-correcciones` y etiqueta `v0.6-c4`.

E8: README, revisión cruzada y entrega

1. Agregue al README.md la sección "Arquitectura C4 (Lab 06)", con el diagrama de contexto en Mermaid (copiado de export/) y enlaces a las imágenes de las cinco vistas.

En la entrega no se hizo: el commit `d8c2792` se llama "README, revisión cruzada y entrega", pero solo modificó tres archivos de `docs/architecture/c4/` (`git show --stat d8c2792`). La sección se agregó en `2a145ac`:

```
## Arquitectura C4 (Lab 06)
Modelo C4 de SismoReporta AQP en [workspace.dsl](docs/architecture/c4/workspace.dsl), ...
### Diagrama de contexto
```mermaid
graph LR
  subgraph diagram ["SismoReporta AQP - Diagrama de contexto (nivel 1)"]
  ...            (copia sin cambios de export/structurizr-Contexto.mmd)
```
Vistas
| Vista | Imagen | C4-PlantUML | Mermaid |
| Contexto | [structurizr-Contexto.png](docs/architecture/c4/img/structurizr-Contexto.png) | ...
  (una fila por cada una de las cinco vistas)
```

```
### Revisión y comunicación
- revision-c4.md, verificar_c4.py, comparacion.md, explicacion.md, bitacora-ia.md
```

Comprobación: los 47 enlaces relativos del `README.md` apuntan a archivos que existen, y el bloque Mermaid se renderizó sin errores con mermaid-cli. La rama también incorpora las correcciones del Lab 05 (`v0.5.1-diseno`), así que el README de `v0.6-c4` ya no tiene los 7 enlaces rotos de la sección del Lab 05.

2. Revisión cruzada.

No se realizó: el laboratorio se desarrolló de forma individual, sin otro grupo con quien intercambiar la revisión. Como sustituto parcial, las vistas pasaron por la lista de revisión oficial (E7), por un segundo revisor con IA cuyos hallazgos se verificaron uno por uno, y por las comprobaciones automáticas de `verificar_c4.py`.

3. Etiqueta de entrega.

El 17/10/26 no se creó ninguna etiqueta para este laboratorio (`git tag` solo lista `v0.5-diseno`, `v0.5.1-diseno`, `v0.7-patrones` y `v0.71-patrones`). La etiqueta `v0.6-c4` se creó el 2026-11-13 sobre el commit de la revisión; su mensaje indica la fecha para no confundirla con la entrega original:

```
git tag -a v0.6-c4 -m "Lab 06: entrega del modelo C4 con las correcciones de la revisión (2026-11-13)"
git push origin lab06-correcciones v0.6-c4

$ git log --oneline -3 v0.6-c4
2a145ac Lab 06: correcciones tras la revisión del informe
4a6a7ca Incorpora las correcciones del Lab 05 (v0.5.1-diseno)
b84689c Lab 05: correcciones tras la revisión del informe
```

II. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Al revisar la entrega del 17/10/26 (commit `d8c2792`) contra la guía y contra lo que afirma el propio informe, se encontraron siete problemas. Todos se resolvieron en el commit `2a145ac` (etiqueta `v0.6-c4`), salvo el que se indica.

Tabla 15. Problemas de la entrega y solución.

#	Problema	Evidencia	Solución
1	Faltaba la sección "Arquitectura C4 (Lab 06)" del README (E8).	`git show --stat d8c2792`: el README no cambia.	Sección con el contexto en Mermaid y las cinco vistas (E8).
2	No había etiqueta de entrega (E8).	`git tag`: ninguna `v0.6`.	Etiqueta anotada `v0.6-c4`.
3	La respuesta 7 del cuestionario decía que el color del servicio de teselas "se corrigió", pero la exportación entregada seguía en azul.	`export/structurizr-Contexto.puml` de `d8c2792`: `System(Serviciodeteselasdemapa, ..., $tags="")`.	Propiedad `"c4plantuml.tags" "true"`; ahora sale gris con "Externo" en la leyenda (Figura 1).
4	`inventario.md` clasificaba el proxy inverso como contenedor, y `workspace.dsl` como infraestructura.	Fila "Proxy inverso / Contenedor" frente a la línea 107 del DSL.	El inventario lo clasifica como nodo de infraestructura.
5	`revision-c4.md` citaba líneas equivocadas y dejaba tres correcciones pendientes (títulos, acrónimos, tecnología del proxy).	Citaba 153 y 163 para vistas que estaban en las líneas 150 y 160.	Líneas corregidas; títulos y acrónimos aplicados. Queda pendiente la tecnología del proxy, que necesita un ADR.
6	La bitácora no tenía fecha, herramienta ni verificación, ni registraba el Prompt IA 1; `explicacion.md` no se había corregido.	Cinco secciones con solo prompt y resultado.	8 interacciones con las seis columnas; dos frases de la explicación corregidas (E7).

7	La afirmación "conteos con un script sobre el JSON" no se podía comprobar.	No había ningún script en el repositorio.	`verificar_c4.py`, con prueba de mutación (E3).

III. SOLUCIÓN DEL CUESTIONARIO

1. ¿Qué problema de los diagramas tradicionales de "cajas y flechas" resuelve C4? Dé un ejemplo de su caso.

En un diagrama libre, cada caja puede significar cualquier cosa; C4 fija qué tipo de pieza es cada una (persona, sistema, contenedor o componente) y en qué nivel de detalle va. En nuestro caso evitó dibujar la "cola de trabajos" como una pieza aparte, cuando en realidad es una tabla dentro de la base de datos (ADR-002), y obligó a decidir que el proxy inverso es infraestructura y no un contenedor.

2. ¿Qué es un "contenedor" en C4 y por qué no equivale a uno de Docker? Dé un ejemplo de su caso.

En C4, un contenedor es una pieza que se ejecuta por separado o un lugar donde se guardan datos; Docker es solo una forma de empaquetar programas. La PWA es un contenedor que corre en el navegador del celular, sin Docker, y en el servidor la API y el worker son dos servicios de systemd, también sin Docker (Tabla 8).

3. ¿Qué público y qué nivel de detalle tiene cada nivel de C4? ¿Por qué rara vez se dibuja el nivel 4 a mano?

Contexto: para todos, muestra el sistema y quién lo usa. Contenedores y componentes: para el equipo técnico, muestran las piezas y sus partes internas. Código: para programadores. El nivel 4 cambia con cada cambio en el código, así que conviene generarlo con herramientas, como hicimos con pyreverse en el Lab 05, en vez de dibujarlo.

4. Compare la vista dinámica de C4 con la secuencia UML de su flujo crítico. ¿Cuándo usaría cada una?

La vista dinámica muestra por qué piezas pasa un reporte, incluidos los almacenes y la transacción del servidor (pasos 4 a 6), y sirve para explicar el recorrido general. La secuencia UML muestra cada operación, los errores (`alt`) y las repeticiones (`loop`), y sirve para programar y probar.

5. ¿Qué ventaja concreta tiene Structurizr DSL frente a dibujar cada vista con C4-PlantUML o Mermaid?

Se describe el sistema una sola vez y todas las vistas salen de esa descripción: al renombrar la API cambia una línea y las cinco vistas se regeneran. Además, la herramienta valida el modelo: detuvo un destino inexistente ("tesela") que PlantUML a mano dibujó sin avisar.

6. ¿Qué elementos de su modelo C4 se justifican con el ADR-001? Cite al menos dos.

La API y el Worker como contenedores separados, porque el ADR-001 decide que el trabajo pesado se haga aparte; los cinco componentes de la API, que son los módulos del monolito modular; y la única base de datos PostgreSQL en el único droplet de la vista de despliegue.

7. Describa un error de la IA en el modelo C4 y cómo lo detectó con la lista de revisión.

Al exportar a C4-PlantUML, el servicio de mapas salió del mismo azul que nuestro sistema, como si lo hubiéramos construido nosotros. Lo detectó la pregunta "¿se entienden los colores?". En la entrega solo quedó registrado; en `v0.6-c4` se corrigió con la propiedad `"c4plantuml.tags" "true"`, y ahora el servicio sale gris y la leyenda dice "Externo".

8. ¿Cómo adaptaría la explicación de su arquitectura para un docente, un inversionista y un nuevo developer?

Al docente le mostraría las decisiones (ADR) y su relación con cada vista; al inversionista, el diagrama de contexto en palabras simples (`explicacion.md`) y el costo de US$ 25,60 al mes; al nuevo developer, los contenedores, los componentes y la vista dinámica del recorrido de un reporte.

IV. CONCLUSIONES

1. Un solo modelo en Structurizr DSL produjo las cinco vistas, sus exportaciones a C4-PlantUML y Mermaid, y un JSON que se puede comprobar con un script. Cuando hubo que corregir colores, títulos y acrónimos, bastó cambiar el modelo y volver a exportar, sin tocar los diagramas generados.
2. El modelo C4 quedó trazable a los laboratorios anteriores: los contenedores salen del ADR-001 al ADR-003, los componentes coinciden con los paquetes del Lab 05 (mismos 5 nombres, 9 dependencias y sin ciclos) y la vista dinámica recorre la misma HU-01 que la secuencia UML.
3. La vista de despliegue sirvió para comprobar una restricción del caso: un droplet con respaldos y un volumen cuestan US$ 25,60 al mes, dentro de los US$ 20-30 de R-03 y sin Docker ni servicios administrados.
4. La IA aceleró el borrador y la revisión, pero cada propuesta necesitó verificación: hubo propuestas corregidas o rechazadas, cinco hallazgos descartados como falsos positivos o exagerados, y dos frases de la explicación que el modelo no respaldaba.
5. La revisión de la entrega mostró que afirmar algo no basta: el README, la etiqueta y la corrección de la pregunta 7 figuraban como hechos sin estarlo. Contrastar cada afirmación con el repositorio (commits, archivos exportados y scripts) fue lo que permitió encontrarlo y corregirlo.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Brown, S. The C4 model for visualising software architecture. https://c4model.com

[2] Brown, S. Structurizr DSL: language reference. https://docs.structurizr.com/dsl/language

[3] Structurizr. Structurizr CLI: validate y export. https://docs.structurizr.com/cli

[4] Ricardo Niepel y colaboradores. C4-PlantUML. https://github.com/plantuml-stdlib/C4-PlantUML

[5] Mermaid. Flowchart syntax. https://mermaid.js.org/syntax/flowchart.html

[6] DigitalOcean. Droplet, Backups y Volumes pricing. https://www.digitalocean.com/pricing

[7] Nygard, M. (2011). Documenting Architecture Decisions. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

[8] Oliva Valdivia, D. (2026). Repositorio cs-2026b-lab04-grupo06, etiqueta v0.6-c4. https://github.com/DavidOliva06/cs-2026b-lab04-grupo06